The Runtime of Geometric Core Subset Sum

Jonathan R. Landers October 28, 2026

Runtime companion to Note VII · Working draft

Purpose. Can we make an enormous search cheaper by finding and reusing structure? This companion defines the geometric core route of Note VII [4], translates its runtime into Note I's certified-coverage accounting, and compares general algorithms. The improvement depends on discovered structure; it supplies no new general worst-case bound.

The problem and the structural opportunity

Given positive integer weights $w_1, \dots, w_N$ and target t, subset sum asks whether

$$\exists x \in \{0,1\}^N : \quad \sum_{i=1}^{N} w_i x_i = t.$$

YES supplies a matching subset; NO excludes every subset. Direct enumeration has 2^N choices. An exact summary can compress this search, but its numeric size must be charged in binary input length.

After exact range, complement and gcd reductions, consider an unresolved kernel $u_1, \dots, u_n$ with target T and binary encoding length L. The route discovers a base $M \geq 2$ and writes $u_i = Mp_i + r_i$, with $p_i \geq 0$ and possibly signed residuals r_i. Retain a core $\mathcal{K}$ of m items with a narrow residual range and put the remaining $e = n - m$ items in E. We use e for the exception count called k in Note VII, to distinguish it from Note I's coverage function.

One core representation, many shifted targets

A core selection has coordinates $q = \sum_{i\in\mathcal{K}} p_i x_i$ and $s = \sum_{i\in\mathcal{K}} r_i x_i$. Define

$$R_- = \sum_{i\in\mathcal{K}} \min(0, r_i), \quad R_+ = \sum_{i\in\mathcal{K}} \max(0, r_i), \quad Q = \min\left(\sum_{i\in\mathcal{K}} p_i, \left\lfloor \frac{T - R_-}{M} \right\rfloor\right).$$

For $Q \geq 0$, the exact reachable-pair table has height $H = Q + 1$ and width $W = R_+ - R_- + 1$. From $(0, 0)$, each item preserves a pair or adds (p_i, r_i). Nonnegative quotients and the full prefix residual strip justify clipping. Multiple subsets may share a pair: HW counts table positions, not original subsets.

For each exceptional subset $A \subseteq E$, solve the shifted core query

$$z_A = T - \sum_{i\in A} u_i, \qquad Mq + s = z_A.$$

The same table serves all 2^e assignments. For example, core $(101, 203)$ and exception 149, with $M = 100$, solve target 250: choosing the exception leaves $z_A = 101$, reached by the core pair $(1, 1)$.

Reject A if z_A lies outside $[0, \sum_{i\in\mathcal{K}} u_i]$, or if this necessary quotient interval is empty:

$$\max\left(0, \left\lceil \frac{z_A - R_+}{M} \right\rceil\right) \leq q \leq \min\left(Q, \left\lfloor \frac{z_A - R_-}{M} \right\rfloor\right).$$

Otherwise test $(q, z_A - Mq)$, at most $G = \min(H, \lfloor (W-1)/M \rfloor + 1)$ pairs per assignment. Interval feasibility alone cannot prove YES. Exhaustive exclusion proves NO; a reachable pair supplies a reconstructed, verified witness.

The paid route cost

Let F pay reductions, acquisition and discovery; B pay enumeration/bounds per assignment; D pay one table construction; Q_{pt} pay a point test; and V pay reconstruction/verification. If I indicates that any assignment survives bounds, then

$$\boxed{T_{\text{route}} \leq F + 2^e B + ID + N_Q Q_{\text{pt}} + V, \qquad N_Q \leq 2^e G.}$$

Build the table only on the first surviving assignment. For the same plan and sufficient resources, $T_{\text{eager}} - T_{\text{lazy}} = (1 - I)D \geq 0$: complete bounds rejection avoids construction; otherwise one table is built and reused.

Replacing the generic functions in Note I

Under Note I's accounting assumptions [5], certified coverage and residual states are

$$C(n) = k(n) + a(n)\delta(n) + p(n), \qquad r(n) = 2^n - C(n),$$

and modeled runtime is

$$T_{\mathrm{I}}(n) = \underbrace{P(n) + Q_{\mathrm{pair}}(n) + \log_2 k(n)}_{\text{discovery, representation and lookup overhead}} + r(n).$$

Here $k(n)$ is structural coverage, $a(n)\delta(n)$ is certified learned elimination, and $p(n)$ is additional pairing coverage. These are counts under that framework, not the new exception count or table size. P pays preprocessing and Q_{pair} pays pairing. The model charges explicit residual search in proportion to the number of uncovered original states.

The cost-level translation. A core query now represents a whole block of original choices. If assignment A makes g_A point tests, with $g_A \le G$, the replacement of accounting roles is

$$\boxed{\begin{aligned} P(n) + Q_{\mathrm{pair}}(n) + \log_2 k(n) &\rightsquigarrow F + ID + V, \\ 2^n - C(n) &\rightsquigarrow \sum_{A \subseteq E} \big(B + g_A Q_{\mathrm{pt}}\big). \end{aligned}}$$

Summing gives $2^e B + N_Q Q_{\mathrm{pt}}$, where $N_Q = \sum_A g_A \le 2^e G$, and hence the boxed route bound on page 1. Early success can stop before this full-enumeration envelope. The arrows specialize the computational roles; they are not algebraic equalities between a state count and a query cost. The route uses no learned gate: exact structure and checks earn its decisions.

Coverage still counts original subsets. Fix an exceptional assignment A. Its block $\mathcal{B}_A$ contains every selection of the m core items, so

$$|\mathcal{B}_A| = 2^m, \qquad 2^n = 2^e 2^m.$$

A certified NO for that assignment excludes the entire block, using bounds or exact reachability. At a stage accounted for solely by j such disjoint NO blocks,

$$C_{\mathrm{blocks}}(n) = j2^m, \qquad r_{\mathrm{states}}(n) = (2^e - j)2^m.$$

Thus 2^e is the number of block queries, not the number of original residual states. A YES witness may stop without excluding all other blocks. The new representation changes how remaining work is processed; it does not invalidate Note I's criterion for its explicit-search model. In particular, neither HW nor $2^n - 2^e$ can simply be substituted for certified coverage.

Bit costs and the sufficient polynomial regime

With word width ω, a conservative construction estimate is

$$D = O((m+1)H\,(1 + \lceil W/\omega \rceil))$$

word operations, apart from scalar arithmetic. Python point tests use integer shifts and also have width-dependent cost. Prefix history for witness reconstruction occupies at most $(m+1)HW$ logical bits, plus row, pointer and interpreter overhead. Logical bit caps are not Python memory measurements.

In bit time, O^* suppressing polynomial factors in binary input length L gives

$$T_{\mathrm{route}} = F + O^*(HW + 2^e GW) \le F + O^*(2^e HW).$$

The first bound exposes one shared construction; the second is a looser shorthand. Neither hides the numeric table dimensions. A sufficient polynomial regime is

$$F = \mathrm{poly}(L), \qquad HW = \mathrm{poly}(L), \qquad e = O(\log L).$$

Indeed $G \le H$ and $2^e = \mathrm{poly}(L)$ bound the query work. Small e alone is insufficient. We do not prove that this detector finds every useful core or that arbitrary binary inputs have compact tables.

Comparison with general subset-sum algorithms

The table compares classical algorithms; n counts the items in the problem being solved. The cited randomized methods have bounded error, whereas our completed decisions are exact. Ordinary O in those results uses their stated RAM model; O^* here suppresses polynomial encoding-length factors.

Method	Time bound	Scope
Horowitz–Sahni [1]	$O^*(2^{n/2})$	General, deterministic exact search.
Chen et al. [2]	$O(2^{n/2}n^{-\gamma})$	Published randomized result; $\gamma > 0.5023$, word/circuit RAM.
October 2026 preprint [3]	Claims $O(2^{0.49n})$	General randomized decision; polynomial-bit inputs, stated word RAM.
Our geometric route	$F + O^*(HW + 2^e GW)$	Exact when completed; depends on discovered core and exceptions.

The October 4 preprint states success probability at least 2/3 per input, with its runtime cap on every execution [3]. It is a recent claim, not an algorithm independently validated or implemented in our experiments. A polynomial-factor saving and a smaller exponential rate are different claims. Target-dependent pseudopolynomial algorithms are another comparison axis: even elementary $O(nT)$ dynamic programming can be preferable for small numerical T, while remaining exponential in the target's bit length.

A sufficient exponential-scale advantage. For polynomial-bit inputs and some fixed $\epsilon > 0$, the coarse structural bound lies below the benchmark scale 2^{cn} if

$$F = O^*(2^{(c-\epsilon)n}), \qquad e + \log_2(HW) \le (c-\epsilon)n.$$

Use $c = 1/2$ for the classical scale, or $c = 0.49$ for the preprint's claimed scale. Polynomial factors in L are then subexponential in n. This sufficient condition compares bounds, not measured speedups or hidden constants; the sharper shared-table expression can allow better regimes.

What the experiments establish, and what remains open

The implementation caps exceptions, table/history size, work and time. It may return *unresolved*: neither YES nor NO has been established within the allowance. Bounded execution therefore does not make it a complete polynomial-time algorithm. A complete MITM fallback preceded by a polynomially budgeted geometric attempt would retain $O^*(2^{n/2})$ worst-case time. This is a possible complete wrapper, not a stronger bound for the capped experimental portfolio.

Note VII reports 453 to 469 completions on 576 fresh targets for core reuse versus matched root geometry. Its later bounds-first refinement avoids 90 of 182 tables; the primary incremental completion comparison is only 953 versus 954, with interval $[0, 3]$, and no clear incremental wall-clock advantage. These are paid-budget results on constructed inputs, not an asymptotic improvement or demonstrated superiority over the general algorithms above.

The research program's concrete progress is an exact, reusable representation whose discovery and checking costs are charged and whose paid completion improves on a matched structural baseline. The next theoretical target is to characterize larger families where cheap discovery guarantees compact HW and few exceptions. Note I's coverage principle remains the organizing test: the representation must earn its cost. This companion adds explanation, not a new experiment.

References

[1] E. Horowitz and S. Sahni. Computing partitions with applications to the knapsack problem. *JACM* 21(2), 277–292, 1974. `https://doi.org/10.1145/321812.321823`.

[2] X. Chen, Y. Jin, T. Randolph and R. A. Servedio. Subset Sum in Time $2^{n/2}/\operatorname{poly}(n)$. *APPROX/RANDOM*, 39:1–39:18, 2023. `https://doi.org/10.4230/LIPIcs.APPROX/RANDOM.2023.39`.

[3] OpenAI. *Subset Sum in Time $O(2^{0.49n})$*. Preprint, October 4, 2026. Manuscript and stated computational model.

[4] J. R. Landers. *Geometric Cores and Exceptions in Subset Sum*. Note VII, working draft, 2026. Main note, proofs and frozen-study evidence.

[5] J. R. Landers. *Certified Coverage of Exponential Search Spaces*. Note I, 2026. Generic framework and coverage criterion.